

Εθνικό και Καποδιστριακό Πανεπιστήμιο Αθηνών
Τμήμα Πληροφορικής και Τηλεπικοινωνιών

Μάθημα: Τεχνολογίες Εφαρμογών Διαδικτύου

Εφαρμογή Διαχείρισης Εκδηλώσεων και Ηλεκτρονικών Κρατήσεων στον Παγκόσμιο Ιστό

Eclass: Ομάδα χρηστών 5
Ονοματεπώνυμο : Παπαθανασίου Ελένη — Αριθμός Μητρώου : 1115202200135
Ονοματεπώνυμο : Τσάκωνας Βασίλης — Αριθμός Μητρώου : 1115202200245

Περιεχόμενα

1 Εισαγωγή **3**
1.1 Αρχιτεκτονική εφαρμογής . . . 3
1.2 Περιεχόμενο Κεφαλαίων . . . 3

2 Παραδοχές **3**
2.1 Μοντέλο Ρόλων: Τρεις ρόλοι(admin, user, guest) . . . 3
2.2 Ασφάλεια και Authentication (JWT) . . . 4
2.3 Διεπαφή Χρήστη (UI/UX) . . . 4
2.4 Διαχείριση Εκδηλώσεων . . . 5
2.5 Κρατήσεις . . . 5
2.6 Σύστημα Μηνυμάτων . . . 6
2.7 Λειτουργίες Admin . . . 6

3 Ρόλοι **7**
3.1 Μοντέλο Ρόλων στο Frontend . . . 7
3.2 Λειτουργίες ανά Ρόλο Χρήστη . . . 7
3.2.1 Επισκέπτης (Guest) . . . 7
3.2.2 Συνδεδεμένος Χρήστης (User) . . . 8
3.2.3 Διαχειριστής (Admin) . . . 9

4 Βάση δεδομένων **9**
4.1 Οντότητες . . . 9
4.2 Σχεδιάγραμμα απεικόνισης οντοτήτων και συσχετίσεων . . . 10

5 Νωτιαίο άκρο (Backend) **11**
5.1 Δομή του backend . . . 11
5.2 REST API . . . 12
5.2.1 Διαδρομές εφαρμογής-Endpoints . . . 12
5.3 Ταυτοποίηση και εξουσιοδότηση . . . 13
5.4 Αναζήτηση και φιλτράρισμα . . . 13

5.4.1 Αναζήτηση και φιλτράρισμα εκδηλώσεων . . . 13
5.5 Εκδηλώσεις . . . 13
5.5.1 Τύπος εισιτηρίων . . . 13
5.5.2 Φωτογραφίες . . . 14
5.5.3 Κρατήσεις . . . 14
5.5.4 Επίλυση προβλήματος συνθηκών ανταγωνισμού . . . 14
5.6 Διαχειριστής . . . 14
5.6.1 Εξαγωγή δεδομένων . . . 14
5.7 Μηνύματα . . . 15

6 Σύστημα συστάσεων **15**
6.1 Βαθμολογίες . . . 16
6.2 Αξιολόγηση . . . 16

7 Μετωπιαίο άκρο (Frontend) **17**
7.1 Δομή του Frontend . . . 17
7.2 Δομή Φακέλων και Αρχείων . . . 17
7.2.1 Root Level . . . 17
7.2.2 Context (src/context/) . . . 17
7.2.3 Components (src/components/) . . . 17
7.2.4 Pages (src/pages/) . . . 18
7.2.5 Services (src/services) . . . 19
7.3 Κεντρικό Routing . . . 19
7.4 Ροή Δεδομένων . . . 19
7.5 Δομή Διαδρομών ανά Σελίδα . . . 20
7.6 Επαναχρησιμοποιήσιμα Components . . . 20
7.7 Διαχείριση Στυλ (CSS) . . . 20

8 Ασφάλεια (TLS/SSL) **21**

9 Εγκατάσταση και εκτέλεση **21**

10 Επίλογος **22**

1 Εισαγωγή

Το αντικείμενο της παρούσας εργασίας είναι η σχεδίαση και η υλοποίηση της εφαρμογής EventHub, μιας εφαρμογής διαχείρισης εκδηλώσεων και ηλεκτρονικών κρατήσεων στον Παγκόσμιο Ιστό. Η εφαρμογή ακολουθεί την αρχιτεκτονική web browser / web server: ο χρήστης έχει πρόσβαση σε αυτήν αποκλειστικά μέσω του προγράμματος περιήγησης παγκόσμιου ιστού, ενώ το σύνολο της επιχειρησιακής λογικής και των δεδομένων βρίσκεται στην πλευρά του εξυπηρετητή.

Κάθε εγγεγραμμένος χρήστης, μετά την έγκριση της αίτησής του από τον διαχειριστή, μπορεί να δημιουργεί δικές του εκδηλώσεις και να πραγματοποιεί κρατήσεις σε εκδηλώσεις άλλων, ενώ ο μη συνδεδεμένος επισκέπτης μπορεί μόνο να πλοηγηθεί και να αναζητήσει.

1.1 Αρχιτεκτονική εφαρμογής

Η εφαρμογή αποτελείται από δύο ανεξάρτητα μέρη που επικοινωνούν αποκλειστικά μέσω HTTP: μια εφαρμογή μίας σελίδας (single-page application) που εκτελείται στον περιηγητή και έναν εξυπηρετητή εφαρμογής. Τα δύο μέρη αναπτύσσονται και εκτελούνται χωριστά, σε διαφορετικά ports (backend:8000, frontend:5173).

Το νωτιαίο άκρο (backend) υλοποιήθηκε σε Django και Django REST Framework, με σχεσιακή βάση δεδομένων PostgreSQL, και εκθέτει το σύνολο της λειτουργικότητας ως προγραμματιστική διεπαφή REST.

Το σχεσιακό μοντέλο απεικονίζεται σε αντικειμενοστραφές μέσω του ORM του Django. Το μετωπιαίο άκρο (frontend) υλοποιήθηκε σε React. Όλη η επικοινωνία με το API περνά από ένα κεντρικό επίπεδο υπηρεσιών πάνω σε ένα κοινό Axios instance, ώστε καμία σελίδα να μη γνωρίζει διευθύνσεις ή επικεφαλίδες.

Η πιστοποίηση της ταυτότητας του χρήστη γίνεται με JSON Web Tokens όπου ο πελάτης στέλνει το access token σε κάθε αίτημα που πραγματοποιεί. Η εξουσιοδότηση ελέγχεται χωριστά για κάθε ενέργεια, ανάλογα με τον ρόλο και τη σχέση του χρήστη με το εκάστοτε αντικείμενο. Τέλος, το σύνολο των αλληλεπιδράσεων, τόσο μεταξύ του περιηγητή και του μετωπιαίου άκρου όσο και μεταξύ του μετωπιαίου και του νωτιαίου άκρου, κρυπτογραφείται μέσω του πρωτοκόλλου TLS.

1.2 Περιεχόμενο Κεφαλαίων

Στο κεφάλαιο 2 παρουσιάζονται οι παραδοχές που έγιναν κατά τη σχεδίαση της εφαρμογής. Στο κεφάλαιο 3 παρουσιάζονται οι ρόλοι που μπορεί να έχει κάποιος στην εφαρμογή και οι ενέργειες που επιτρέπονται στον καθένα. Στο κεφάλαιο 4 παρουσιάζεται η σχεδίαση της βάσης δεδομένων. Στο κεφάλαιο 5 περιγράφεται το νωτιαίο άκρο: η οργάνωση του κώδικα, η διεπαφή REST, η διαχείριση των εκδηλώσεων και των κρατήσεων και η εξαγωγή δεδομένων σε XML και JSON. Το κεφάλαιο 6 είναι αφιερωμένο στο σύστημα συστάσεων, το οποίο αναπτύχθηκε με τη χρήση του αλγορίθμου Biased Matrix Factorization, υλοποιημένου εκ του μηδενός. Στο κεφάλαιο 7 περιγράφεται το μετωπιαίο άκρο. Το κεφάλαιο 8 αφορά την ασφάλεια και το πρωτόκολλο TLS/SSL. Στο κεφάλαιο 9 δίνονται οι οδηγίες εγκατάστασης και εκτέλεσης. Τέλος, στο κεφάλαιο 10 παρατίθεται ο επίλογος της εργασίας.

2 Παραδοχές

2.1 Μοντέλο Ρόλων: Τρεις ρόλοι(admin, user, guest)

Η σημαντικότερη αρχιτεκτονική παραδοχή της εργασίας αφορά το μοντέλο ρόλων των χρηστών. Ενώ η εκφώνηση ορίζει τέσσερις ρόλους - Διαχειριστής Εφαρμογής, Διοργανωτής, Συμμετέχων και Επισκέπτης - στο μετωπιαίο άκρο της εφαρμογής υλοποιήθηκε μοντέλο τριών ρόλων συνδυάζοντας τον organizer και τον attendee:

Ρόλος Frontend	Πηγή	Περιγραφή
Guest/Visitor	Μη συνδεδεμένος χρήστης	Περιήγηση και αναζήτηση εκδηλώσεων
User	Συνδεδεμένος με is_staff=false	Διοργανωτής και Συμμετέχων ταυτόχρονα
Admin	Συνδεδεμένος με is_staff=true	Ό,τι ο User και ενέργειες διαχείρισης

Οι ρόλοι Διοργανωτής και Συμμετέχων δεν διαχωρίζονται στο μετωπιαίο άκρο, επειδή:

1. Κάθε εγγεγραμμένος χρήστης μπορεί ταυτόχρονα να δημιουργεί εκδηλώσεις (ως διοργανωτής) και να κάνει κρατήσεις σε εκδηλώσεις άλλων χρηστών (ως συμμετέχων). Οι δύο ιδιότητες δεν είναι αμοιβαία αποκλειόμενες και δεν απαιτούν διαφορετικά δικαιώματα πρόσβασης.

2. Ο διαχωρισμός θα απαιτούσε επιπλέον μηχανισμό εναλλαγής ρόλου στο UI (π.χ. toggle ή ξεχωριστό μενού), χωρίς ουσιαστικό λειτουργικό όφελος, καθώς όλες οι ενέργειες παραμένουν διαθέσιμες στον ίδιο χρήστη.

3. Η διάκριση επιτυγχάνεται μέσω των tabs του UserDashboard: τα tabs My Events και Reservations εξυπηρετούν τις ανάγκες του διοργανωτή, ενώ το tab My Bookings τις ανάγκες του συμμετέχοντα. Ο χρήστης μεταβαίνει ο ίδιος στο κατάλληλο tab ανάλογα με το τι θέλει να κάνει.

4. Το backend χρησιμοποιεί το flag is_staff ή is_superuser ως μοναδική πηγή διάκρισης μεταξύ απλού χρήστη και διαχειριστή.

Η παραδοχή αυτή απλοποιεί σημαντικά τη διαχείριση κατάστασης (state management), μειώνει την πολυπλοκότητα του routing και αποφεύγει την περιττή επικάλυψη UI.

2.2 Ασφάλεια και Authentication (JWT)

Για τη διαχείριση της ταυτότητας των χρηστών υιοθετήθηκαν οι ακόλουθες παραδοχές:

1. Αποθήκευση tokens σε sessionStorage αντί localStorage. Τα JWT tokens (access και refresh) αποθηκεύονται στο sessionStorage του browser, γεγονός που συνεπάγεται ότι διαγράφονται αυτόματα με το κλείσιμο του tab. Η επιλογή αυτή αυξάνει την ασφάλεια, καθώς περιορίζει το χρονικό παράθυρο στο οποίο ένα token παραμένει διαθέσιμο σε περίπτωση μη εξουσιοδοτημένης πρόσβασης στη συσκευή.

2. Σταθερή διάρκεια tokens. Το access token έχει διάρκεια 10 λεπτών και το refresh token 1 ημέρας. Η επιλογή σύντομης διάρκειας για το access token μειώνει την έκθεση σε περίπτωση διαρροής, ενώ το refresh token επιτρέπει τη συνέχιση της συνεδρίας χωρίς επαναληπτική σύνδεση.

3. Αυτόματη ανανέωση μέσω interceptor. Η ανανέωση του access token πραγματοποιείται αυτόματα από τον Axios interceptor (src/services/client.js) χωρίς να απαιτείται η παρέμβαση του χρήστη. Αυτό εξασφαλίζει ομαλή εμπειρία χρήσης ακόμη και σε μεγάλης διάρκειας συνεδρίες.

4. Καθαρισμός tokens σε λήξη refresh token. Αν λήξει και το refresh token, τα tokens διαγράφονται και εκπέμπεται event auth-expired, το οποίο ενημερώνει το UI ώστε να αποσυνδεθεί ο χρήστης.

2.3 Διεπαφή Χρήστη (UI/UX)

Οι ακόλουθες παραδοχές αφορούν τον τρόπο παρουσίασης και αλληλεπίδρασης με τον χρήστη:

1. Επιβεβαίωση κρίσιμων ενεργειών μέσω window.confirm(). Για ενέργειες που δεν μπορούν να αναιρεθούν, όπως κράτηση εισιτηρίου, διαγραφή εκδήλωσης, ακύρωση εκδήλωσης, μαζική διαγραφή, χρησιμοποιείται ο εγγενής διάλογος window.confirm() του browser αντί για custom modal. Η επιλογή αυτή καλύπτει την απαίτηση για «υποχρεωτική επιβεβαίωση» με απλό, αξιόπιστο και cross-browser συμβατό τρόπο.

2. Σταθερή μορφή ημερομηνιών σε 24ωρη βάση. Όλες οι ημερομηνίες εμφανίζονται με τη μορφή DD/MM/YYYY, HH:mm ανεξαρτήτως locale του χρήστη, ώστε να διασφαλίζεται συνέπεια στην παρουσίαση και να αποφεύγονται παρανοήσεις μεταξύ 12ωρης και 24ωρης μορφής. Η μορφοποίηση υλοποιείται κεντρικά μέσω helper functions.

3. Σελίδα καλωσορίσματος χωρίς Navbar/Footer. Η αρχική σελίδα (WelcomePage) παρουσιάζεται σε πλήρη οθόνη χωρίς τα κοινά στοιχεία πλοήγησης, ώστε να λειτουργεί ως αυτόνομο εισαγωγικό σημείο. Η μετάβαση στην κύρια εφαρμογή γίνεται με κλικ οπουδήποτε στη σελίδα ή με τα πλήκτρα Enter/Space.

4. Loading indicators. Σε κάθε ασύγχρονη λειτουργία (π.χ. φόρτωση δεδομένων, υποβολή φόρμας, ανανέωση auth state) εμφανίζεται ενιαίος spinner (LoadingSpinner), ώστε ο χρήστης να έχει σαφή οπτική ανατροφοδότηση.

2.4 Διαχείριση Εκδηλώσεων

Οι ακόλουθες παραδοχές αφορούν τη ροή δημιουργίας, επεξεργασίας και διαχείρισης εκδηλώσεων από τον διοργανωτή:

1. Wizard τριών βημάτων για Create/Edit Event. Η δημιουργία και η επεξεργασία εκδήλωσης πραγματοποιείται μέσω οδηγού (wizard) τριών βημάτων:

   - Βήμα 1: Event Details (τίτλος, τύπος, τοποθεσία, ημερομηνίες, κατηγορίες, χωρητικότητα).
   - Βήμα 2: Ticket Types (όνομα, τιμή, ποσότητα ανά τύπο).
   - Βήμα 3: Photos & Description.

   Πριν από την τελική υποβολή, εμφανίζεται ενδιάμεση σελίδα Preview στην οποία ο χρήστης μπορεί να ελέγξει το σύνολο των στοιχείων και να επιστρέψει σε οποιοδήποτε βήμα για διόρθωση.

2. Έλεγχος χωρητικότητας (capacity validation) σε πραγματικό χρόνο. Στο βήμα 2 εμφανίζεται live δείκτης Capacity / Tickets allocated / Remaining, ο οποίος ενημερώνεται καθώς ο χρήστης μεταβάλλει τις ποσότητες των τύπων εισιτηρίων. Δεν επιτρέπεται η υποβολή αν το άθροισμα των ποσοτήτων υπερβαίνει τη συνολική χωρητικότητα.

3. Διαχείριση διαθέσιμων εισιτηρίων από το backend. Το frontend αποστέλλει μόνο την αρχική ποσότητα (quantity) κάθε τύπου εισιτηρίου. Το πεδίο available υπολογίζεται αποκλειστικά από το backend μετά από κάθε κράτηση, ώστε να διασφαλίζεται η συνέπεια των δεδομένων ακόμη και σε περιπτώσεις ταυτόχρονων κρατήσεων.

4. Διαγραφή έναντι Ακύρωσης. Διακρίνονται σαφώς δύο ενέργειες:

   - Διαγραφή: Επιτρέπεται μόνο όταν η εκδήλωση βρίσκεται σε κατάσταση DRAFT ή όταν δεν έχει πραγματοποιηθεί καμία κράτηση (total_booked $===$ 0).
   - Ακύρωση: Αφορά μόνο δημοσιευμένες (PUBLISHED) εκδηλώσεις. Το UI εμφανίζει modal επιβεβαίωσης και ενημερώνει τον χρήστη ότι οι συμμετέχοντες με ενεργές κρατήσεις θα ειδοποιηθούν αυτόματα μέσω μηνύματος.

5. Περιορισμός φωτογραφιών. Κάθε εκδήλωση μπορεί να έχει έως 20 φωτογραφίες, τύπων image/jpeg, image/png και image/webp. Η προεπισκόπηση γίνεται τοπικά μέσω URL.createObjectURL πριν από τη μεταφόρτωση.

6. Upload φωτογραφιών σε ξεχωριστό βήμα. Οι φωτογραφίες μεταφορτώνονται μετά τη δημιουργία της εκδήλωσης, μέσω ξεχωριστών κλήσεων στο endpoint /events/id/photos/. Η παραδοχή αυτή οφείλεται στον τρόπο με τον οποίο είναι σχεδιασμένο το backend API (δεν υποστηρίζει multipart στη δημιουργία εκδήλωσης).

2.5 Κρατήσεις

Οι ακόλουθες παραδοχές αφορούν τη ροή κράτησης εισιτηρίων:

1. Έλεγχος εγκυρότητας πριν την υποβολή (booking guard). Πριν από την αποστολή του POST για τη δημιουργία κράτησης, το frontend ελέγχει ξανά:

   - status $===$ "PUBLISHED": η εκδήλωση είναι ενεργή,
   - available_capacity $>$ 0: υπάρχουν διαθέσιμες θέσεις
   - end_datetime $>$ now: η εκδήλωση δεν έχει τελειώσει

   Ο έλεγχος αυτός λειτουργεί ως επιπλέον δικλείδα ασφαλείας πέραν των ελέγχων του backend.

2. Προσωρινή αποθήκευση draft κράτησης. Κατά τη μετάβαση από τη σελίδα λεπτομερειών εκδήλωσης στη σελίδα ολοκλήρωσης κράτησης, τα δεδομένα (event ID, ticket type, quantity) αποθηκεύονται προσωρινά στο localStorage με χρονική ισχύ 10 λεπτών. Έτσι διατηρούνται σε περίπτωση ανανέωσης της σελίδας.

3. Φίλτρα κρατήσεων χωρίς pagination. Στη σελίδα My Bookings εφαρμόζονται client-side φίλτρα (All / Confirmed / Cancelled) χωρίς σελιδοποίηση, καθώς ο αναμενόμενος όγκος κρατήσεων ανά χρήστη είναι μικρός και δεν δικαιολογεί server-side pagination.

4. Επιβεβαίωση με αναλυτικό μήνυμα. Πριν από την οριστικοποίηση της κράτησης, εμφανίζεται window.confirm() με αναλυτικό μήνυμα που περιλαμβάνει: τίτλο εκδήλωσης, τύπο εισιτηρίου, ποσότητα και συνολικό κόστος, καθώς και προειδοποίηση ότι η ενέργεια δεν μπορεί να αναιρεθεί.

2.6 Σύστημα Μηνυμάτων

Για τη λειτουργία της επικοινωνίας μεταξύ διοργανωτή και συμμετέχοντα, υιοθετήθηκαν οι ακόλουθες παραδοχές:

1. Περιορισμός εκκίνησης συνομιλίας. Ένας χρήστης μπορεί να ξεκινήσει νέα συνομιλία μόνο εφόσον υπάρχει σχέση με συγκεκριμένη εκδήλωση:

   - ως συμμετέχων που έχει κάνει κράτηση (μήνυμα προς τον διοργανωτή), ή
   - ως διοργανωτής που έχει λάβει κρατήσεις (μήνυμα προς κάποιον συμμετέχοντα).

   Αυτό αποτρέπει ανεπιθύμητα μηνύματα και διατηρεί τη συζήτηση συνδεδεμένη με το πλαίσιο της εκδήλωσης.

2. Οργάνωση σε Inbox/Sent. Τα μηνύματα οργανώνονται σε δύο φακέλους (Inbox / Sent) με δυνατότητα αναζήτησης βάσει ονόματος συνομιλητή ή τίτλου εκδήλωσης.

3. Ένδειξη μη-αναγνωσμένων. Ο αριθμός των μη-αναγνωσμένων μηνυμάτων εμφανίζεται τόσο στο Navbar όσο και στο tab «Messages» του dashboard. Η ένδειξη μη-αναγνωσμένων μηνυμάτων ανανεώνεται κάθε 20 δευτερόλεπτα.

4. Προβολή ολόκληρης της συνομιλίας. Κάθε εγγραφή στη λίστα με τα μηνύματα αντιστοιχεί σε συνομιλία (conversation). Με το άνοιγμά της, στο πλαϊνό panel φορτώνεται και εμφανίζεται ολόκληρο το ιστορικό, δηλαδή όλα τα μηνύματα που έχουν ανταλλάξει οι δύο χρήστες, ανεξάρτητα από τον φάκελο (Inbox/Sent) από τον οποίο ξεκίνησε η προβολή. Τα μηνύματα εμφανίζονται ως chat bubbles με ημερομηνία/ώρα κάτω από κάθε ένα, και το panel κάνει auto-scroll στο τελευταίο μήνυμα. Παράλληλα, με το άνοιγμα σημαίνονται όλα τα εισερχόμενα της συνομιλίας ως αναγνωσμένα και ενημερώνεται το unread badge. Ο χρήστης μπορεί να απαντήσει στο ίδιο conversation (η απάντηση προστίθεται άμεσα στο thread χωρίς επαναφόρτωση), να κλείσει το panel, ή να διαγράψει ολόκληρη τη συνομιλία.

5. Διαγραφή μηνύματος και συνομιλίας. Ο χρήστης μπορεί να διαγράψει μεμονωμένα μηνύματα ή ολόκληρη την συνομιλία. Η διαγραφή ακολουθεί τη λογική (soft-delete) και αφορά μόνο τον χρήστη που τη ζητά, χωρίς να επηρεάζει το αντίγραφο του άλλου συμμετέχοντα. Αυτό υλοποιείται μέσω δύο ζευγών flags: για τη συνομιλία deleted_by_attendee, deleted_by_organizer και για το μήνυμα deleted_by_sender, deleted_by_receiver. Αν μόνο ο ένας χρήστης διαγράψει το μήνυμα τότε το flag αλλάζει τιμή και το διαγεγραμμένο μήνυμα δεν φαίνεται σε αυτόν που το διέγραψε, ενώ στον άλλον φαίνεται κανονικά. Σε περίπτωση που και οι δύο χρήστες επιλέξουν ένα μήνυμα ή μια συνομιλία να διαγραφεί, τότε υπάρχει οριστική διαγραφή από τη βάση.

2.7 Λειτουργίες Admin

Οι ακόλουθες παραδοχές αφορούν τις λειτουργίες του διαχειριστή:

1. Redirect μετά το login. Ο διαχειριστής, μετά την επιτυχή σύνδεση, ανακατευθύνεται απευθείας στο /profile?tab=users (User Management), το οποίο αποτελεί την κύρια λειτουργία του.

2. Καθολική πρόσβαση (bypass). Ο διαχειριστής περνάει από κάθε έλεγχο ρόλου στο RequireRole, χωρίς περιορισμούς.

3. Client-side φιλτράρισμα εξαγωγών. Το φιλτράρισμα ανά κατάσταση (ALL / PUBLISHED / DRAFT / COMPLETED / CANCELLED) στις εξαγωγές XML/JSON πραγματοποιείται στο frontend (parsing του πλήρους export που επιστρέφει το backend). Αυτό οφείλεται στο ότι το backend export endpoint δεν δέχεται παράμετρο status.

4. Κριτήρια εξαγωγής εκδήλωσης. Μια εκδήλωση εξάγεται μόνο αν διαθέτει τουλάχιστον μία κατηγορία και έναν τύπο εισιτηρίου, όπως απαιτεί το DTD της εκφώνησης.

5. Έγκριση χρηστών μόνο σε PENDING. Οι ενέργειες Approve/Reject είναι διαθέσιμες μόνο για χρήστες με approval_status === "PENDING". Χρήστες ήδη εγκεκριμένοι ή απορριφθέντες δεν μπορούν να μεταβάλουν κατάσταση.

6. Ισοτιμία διαχειριστών. Δεν υπάρχει διάκριση «Super Admin» - όλοι οι χρήστες με is_staff = true έχουν τα ίδια δικαιώματα.

3 Ρόλοι

Στην εφαρμογή EventHub διακρίνονται τρεις (3) κατηγορίες χρηστών, οι οποίες καθορίζουν το επίπεδο πρόσβασης και τις διαθέσιμες λειτουργίες. Στην παρούσα ενότητα παρουσιάζεται το μοντέλο ρόλων που υιοθετήθηκε, αναλύονται οι λειτουργίες κάθε ρόλου και περιγράφεται ο τεχνικός μηχανισμός ελέγχου πρόσβασης. Οι σχεδιαστικές παραδοχές που διέπουν το μοντέλο ρόλων έχουν τεκμηριωθεί στην Ενότητα 2 (Παραδοχές), ενώ η αντιστοίχιση κάθε λειτουργίας με τα αρχεία υλοποίησης παρουσιάζεται στην Ενότητα 7 (Frontend).

3.1 Μοντέλο Ρόλων στο Frontend

Στο μετωπιαίο άκρο της εφαρμογής υλοποιήθηκε μοντέλο τριών (3) κατηγοριών χρηστών:

Ρόλος Frontend	Πηγή	Περιγραφή
Guest/Visitor	Μη συνδεδεμένος χρήστης	Περιήγηση και αναζήτηση εκδηλώσεων
User	Συνδεδεμένος με is_staff=false	Διοργανωτής και Συμμετέχων ταυτόχρονα
Admin	Συνδεδεμένος με is_staff=true	Ό,τι ο User και ενέργειες διαχείρισης

Ο διαχωρισμός μεταξύ Guest, User και Admin πραγματοποιείται κεντρικά μέσω του component RequireRole, το οποίο ελέγχει:

- Αν ο χρήστης είναι συνδεδεμένος (isAuthenticated).
- Αν ο ρόλος του (USER ή ADMIN, με βάση το flag is_staff) περιλαμβάνεται στη λίστα των επιτρεπόμενων ρόλων για τη συγκεκριμένη διαδρομή.

Ο Admin έχει καθολική πρόσβαση σε όλες τις διαδρομές (bypass), ενώ ο User περιορίζεται στις διαδρομές που απαιτούν απλή ταυτοποίηση. Ο Guest ανακατευθύνεται αυτόματα στο /signin όταν επιχειρήσει να προσπελάσει προστατευμένη διαδρομή.

3.2 Λειτουργίες ανά Ρόλο Χρήστη

3.2.1 Επισκέπτης (Guest)

Ο Επισκέπτης είναι χρήστης που δεν έχει συνδεθεί στην εφαρμογή. Έχει πρόσβαση σε όλες τις δημόσιες σελίδες, αλλά όχι σε λειτουργίες που απαιτούν ταυτοποίηση.

Προσβάσιμες διαδρομές: /, /homepage, /events, /events/details/:id, /signin, /signup

Διαθέσιμες ενέργειες:

1. Προβολή σελίδας καλωσορίσματος.
2. Μετάβαση στην αρχική σελίδα (με κλικ οπουδήποτε ή με Enter/Space).

3. Εγγραφή νέου λογαριασμού.
4. Σύνδεση σε υπάρχοντα λογαριασμό.
5. Περιήγηση εκδηλώσεων.
6. Αναζήτηση με φίλτρα: κατηγορία, περιοχή, ημερομηνία, εύρος τιμής, ελεύθερο κείμενο.
7. Σελιδοποίηση αποτελεσμάτων (30 ανά σελίδα).
8. Προβολή αναλυτικών στοιχείων εκδήλωσης.

Περιορισμοί:

- Δεν μπορεί να πραγματοποιήσει κράτηση. Το κουμπί «Tickets» στη σελίδα λεπτομερειών τον ανακατευθύνει στο /signin.
- Δεν του εμφανίζεται ο σύνδεσμος «Recommended Events» στο Navbar.
- Δεν έχει πρόσβαση σε /profile ή σε οποιαδήποτε σελίδα διαχείρισης.

3.2.2 Συνδεδεμένος Χρήστης (User)

Ο Συνδεδεμένος Χρήστης αποτελεί τη βασική κατηγορία χρήστη της εφαρμογής, καθώς συνδυάζει τους ρόλους του Διοργανωτή και του Συμμετέχοντα.

Προσβάσιμες διαδρομές: Όλες οι δημόσιες διαδρομές και επιπλέον /profile, /profile/edit, /profile/messages/new, /profile/events/*, /profile/reservations/*, /events/details/:id/booking, /recommendations

Γενικές λειτουργίες:

1. Προβολή προσωπικών πληροφοριών
2. Επεξεργασία προσωπικών πληροφοριών

Λειτουργίες ως Συμμετέχων:

1. Επιλογή τύπου και ποσότητας εισιτηρίων.
2. Κράτηση εισιτηρίων με υποχρεωτική επιβεβαίωση.
3. Προβολή προσωπικών κρατήσεων με φίλτρα (All / Confirmed / Cancelled).
4. Αποστολή μηνύματος σε διοργανωτή.
5. Προβολή προτεινόμενων εκδηλώσεων.
6. Ανάγνωση και αποστολή μηνυμάτων.

Λειτουργίες ως Διοργανωτής:

1. Δημιουργία εκδήλωσης μέσω οδηγού τριών βημάτων με ενδιάμεση προεπισκόπηση.
2. Επεξεργασία υπάρχουσας εκδήλωσης.
3. Διαγραφή εκδήλωσης (μόνο σε κατάσταση Draft ή χωρίς κρατήσεις).
4. Ακύρωση δημοσιευμένης εκδήλωσης.
5. Δημοσίευση εκδήλωσης ή αποθήκευση ως Draft.
6. Προβολή κρατήσεων ανά εκδήλωση.
7. Μαζική διαγραφή εκδηλώσεων.
8. Αποστολή μηνύματος σε συμμετέχοντα.

3.2.3 Διαχειριστής (Admin)

Ο Διαχειριστής αποτελεί την κατηγορία χρήστη με τα περισσότερα δικαιώματα. Πέραν των λειτουργιών του απλού χρήστη, έχει πρόσβαση σε εργαλεία διαχείρισης χρηστών, εκδηλώσεων και εξαγωγής δεδομένων.

Προσβάσιμες διαδρομές: Όλες οι δημόσιες και οι διαδρομές User, και επιπλέον: /profile?tab=users, /profile?tab=events, /profile?tab=export

Διαθέσιμες Ενέργειες:

1. Προβολή όλων των εγγεγραμμένων χρηστών.
2. Φιλτράρισμα χρηστών (All / Pending / Approved / Rejected).
3. Αναζήτηση χρηστών βάσει username, email ή ονόματος.
4. Έγκριση ή απόρριψη αίτησης εγγραφής.
5. Προβολή όλων των εκδηλώσεων της εφαρμογής.
6. Ακύρωση ή διαγραφή εκδήλωσης.
7. Εξαγωγή εκδηλώσεων σε μορφή XML ή JSON.
    - Εξαγωγή όλων των εκδηλώσεων.
    - Εξαγωγή μεμονωμένων εκδηλώσεων ανάλογα την κατάσταση της διοργάνωσης (DRAFT, PUBLISHED, COMPLETED, CANCELLED),
    - Εξαγωγή μεμονωμένων εκδηλώσεων ενός συγκεκριμένου διοργανωτή (μέσω της καρτέλας User Management)
    - Εξαγωγή μιας συγκεκριμένης εκδήλωσης.

4 Βάση δεδομένων

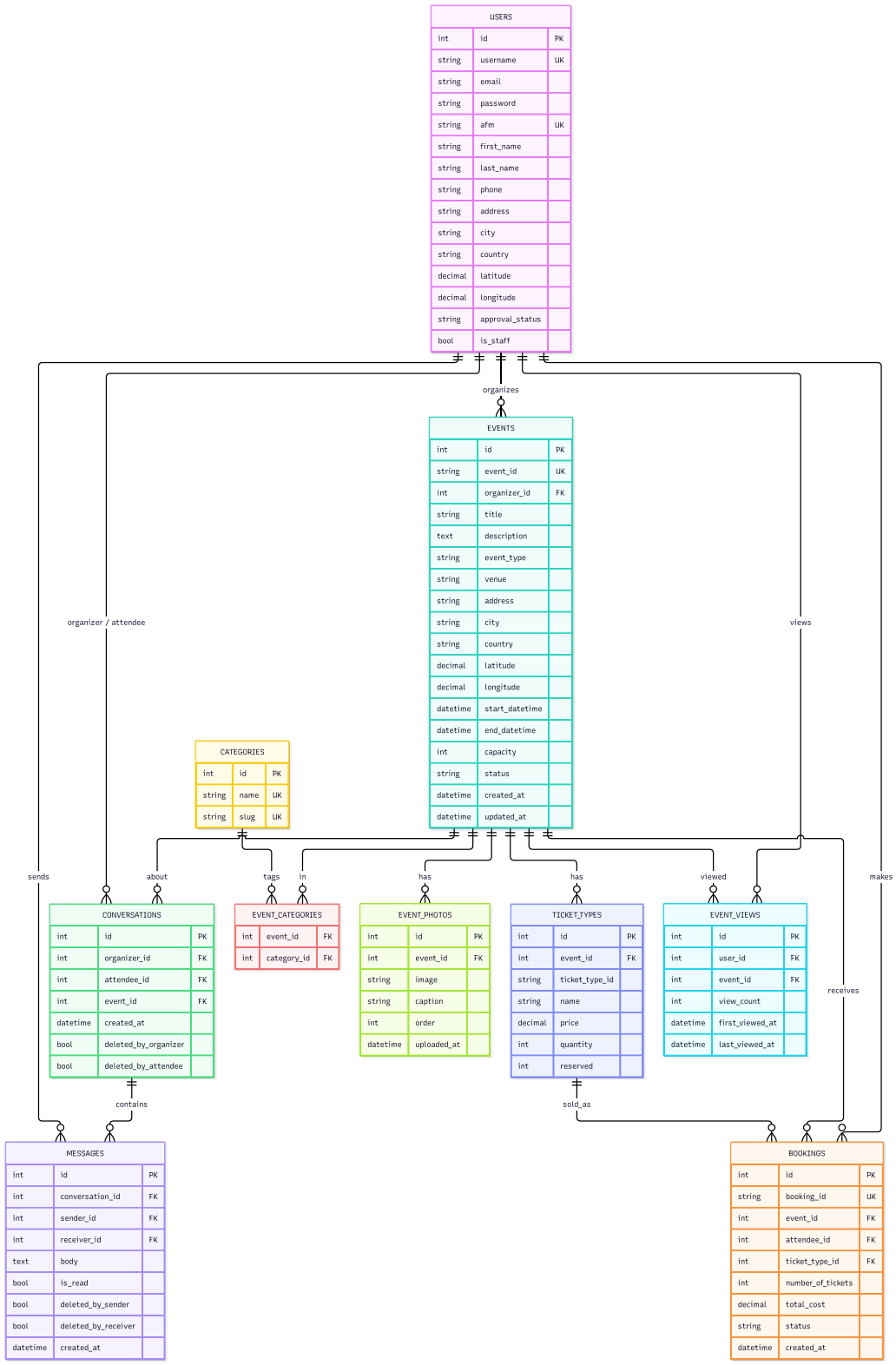
Η βάση δεδομένων της εφαρμογής είναι σχεσιακή (PostgreSQL) και σχεδιάστηκε με βάση την περιγραφή της εκφώνησης. Το σχεσιακό μοντέλο απεικονίζεται σε αντικειμενοστραφές μέσω του ORM(Object Relational Mapping) του Django: κάθε οντότητα αντιστοιχεί σε μία κλάση μοντέλου και κάθε στιγμιότυπο σε μία γραμμή πίνακα. Το σχήμα παράγεται αποκλειστικά από τα μοντέλα, μέσω του μηχανισμού μεταναστεύσεων (migrations), ώστε ο ορισμός των πινάκων να υπάρχει σε ένα μόνο σημείο και να μη μπορεί να αποκλίνει από τον κώδικα.

4.1 Οντότητες

Στον παρακάτω πίνακα παρουσιάζονται οι οντότητες που έχουν σχεδιαστεί:

Οντότητα	Περιγραφή
User	Χρήστες
Event	Εκδηλώσεις
Category	Κατηγορίες στις οποίες μπορεί να ανήκει μια εκδήλωση
EventCategory	Συνδετικός πίνακας μεταξύ εκδηλώσεων και κατηγοριών (σχέση N:N)
TicketType	Εισιτήρια της εκάστοτε εκδήλωσης
EventPhoto	Φωτογραφίες της εκάστοτε εκδήλωσης
Booking	Κρατήσεις
Conversation	Συνομιλίες
Message	Μηνύματα
EventView	Προβολές κάθε εκδήλωσης (για το σύστημα συστάσεων)

4.2 Σχεδιάγραμμα απεικόνισης οντοτήτων και συσχετίσεων



Διάγραμμα οντοτήτων-συσχετίσεων της βάσης δεδομένων

5 Νωτιαίο άκρο (Backend)

5.1 Δομή του backend

Ο πηγαίος κώδικας του νωτιαίου άκρου είναι οργανωμένος σε 6 βασικούς φακέλους, ακολουθώντας τη σύμβαση του Django και διαχωρίζοντας τις αρμοδιότητες ανάλογα με το είδος του κώδικα:

```
core/
├── management/
├── migrations/
├── models/
├── serializers/
├── views/
├── services/
├── admin.py
├── filters.py
├── urls.py
├── apps.py
├── permissions.py
└── utils.py
```

Ρόλος κάθε φακέλου και αρχείου:

- **management**/: Περιέχει εντολές που εκτελούνται από τη γραμμή εντολών μέσω του `manage.py`, εκτός του κύκλου των αιτήσεων HTTP. Εδώ βρίσκεται η εντολή `evaluate_recommender`, η οποία εκπαιδεύει και αξιολογεί το σύστημα συστάσεων.
- **migrations**/: Περιέχει τα αρχεία που παράγουν το σχήμα της βάσης καθώς και δύο μεταναστεύσεις δεδομένων που αρχικοποιούν τις κατηγορίες και ενσωματώνουν έναν χρήστη με ρόλο διαχειριστή.
- **models**/: Περιέχει τις οντότητες της εφαρμογής.
- **serializers**/: Περιέχει τη μετατροπή μεταξύ αντικειμένων και JSON, μαζί με την επικύρωση των εισερχόμενων δεδομένων.
- **views**/: Περιέχει τη διαχείριση των αιτήσεων HTTP, τον έλεγχο εξουσιοδότησης και τη σελιδοποίηση.
- **services**/: Περιέχει την επιχειρησιακή λογική που αφορά περισσότερες από μια οντότητες.
- **admin.py**: Περιέχει τη δήλωση των μοντέλων στη διεπαφή διαχείρισης του Django και τις προσαρμογές κάθε πίνακα.
- **filters.py**: Περιέχει τον δηλωτικό ορισμό των παραμέτρων αναζήτησης της λίστας εκδηλώσεων.
- **urls.py**: Περιέχει την αντιστοίχιση διευθύνσεων σε όψεις: τον δρομολογητή του DRF για τα ViewSets και ρητές διαδρομές για την ταυτοποίηση και την εξαγωγή δεδομένων.
- **apps.py**: Περιέχει την κλάση ρυθμίσεων της εφαρμογής.
- **permissions.py**: Περιέχει τις κλάσεις εξουσιοδότησης, οι οποίες ελέγχουν αν ένας χρήστης δικαιούται μια ενέργεια.
- **utils.py**: Περιέχει βοηθητικές συναρτήσεις κοινής χρήσης.

5.2 REST API

Η προγραμματιστική διεπαφή σχεδιάστηκε με προσανατολισμό στους πόρους: κάθε οντότητα αντιστοιχεί σε μία διεύθυνση και η ενέργεια δηλώνεται από τη μέθοδο HTTP και όχι από τη διεύθυνση. Έτσι η ίδια διεύθυνση /api/events/ εξυπηρετεί την ανάκτηση της λίστας με GET και τη δημιουργία με POST, ενώ η /api/events/id/ εξυπηρετεί την ανάγνωση, την τροποποίηση και τη διαγραφή ενός συγκεκριμένου στιγμιοτύπου. Όλες οι διαδρομές βρίσκονται κάτω από το πρόθεμα /api/ και όλα τα μηνύματα, σε αμφότερες τις κατευθύνσεις, είναι JSON. Οι διαδρομές δεν γράφονται χειροκίνητα: τα ViewSets δηλώνονται σε έναν DefaultRouter του DRF. Πέρα από τις τυπικές διαδρομές, υπάρχουν και ορισμένες ενέργειες μετάβασης κατάστασης. Αυτές εκτίθενται ως χωριστές διαδρομές ενέργειας (@action).

5.2.1 Διαδρομές εφαρμογής-Endpoints

Τα endpoints της εφαρμογής παρουσιάζονται στον παρακάτω πίνακα:

Διαδρομή	HTTP method	Λειτουργία
/api/auth/register/	POST	Εγγραφή νέου χρήστη σε κατάσταση αναμονής
/api/auth/login/	POST	Έκδοση ζεύγους tokens
/api/auth/refresh/	POST	Ανανέωση του access token
/api/auth/me/	GET, PATCH	Ανάκτηση και ενημέρωση του προφίλ
/api/events/	GET, POST	Λίστα και δημιουργία εκδηλώσεων
/api/events/{id}/	GET	Ανάκτηση στοιχείων συγκεκριμένης εκδήλωσης
/api/events/{id}/	PATCH	Αλλαγή στοιχείων συγκεκριμένης εκδήλωσης
/api/events/{id}/	DELETE	Διαγραφή εκδήλωσης
/api/events/{id}/publish/	POST	Δημοσίευση προσχεδίου
/api/events/{id}/cancel/	POST	Ακύρωση δημοσιευμένης εκδήλωσης
/api/events/{id}/photos/	POST	Προσθήκη φωτογραφίας
/api/events/{id}/photos/{photo_id}/	DELETE	Αφαίρεση φωτογραφίας
/api/events/recommendations/	GET	Προσωποποιημένες συστάσεις
/api/ticket-types/	POST	Δημιουργία τύπων εισιτηρίων
/api/ticket-types/{id}/	PATCH	Τροποποίηση τύπου εισιτηρίου
/api/ticket-types/{id}/	DELETE	Διαγραφή τύπου εισιτηρίου
/api/ticket-types/bulk/	POST	Μαζική δημιουργία τύπων κατά τη δημιουργία εκδήλωσης
/api/bookings/	GET	Κρατήσεις μιας εκδήλωσης για τον διοργανωτή
/api/bookings/	POST	Δημιουργία κράτησης
/api/bookings/me/	GET	Οι κρατήσεις του συνδεδεμένου χρήστη
/api/bookings/{id}/confirm/	POST	Οριστικοποίηση κράτησης
/api/conversations/	GET	Οι συνομιλίες του χρήστη
/api/conversations/{id}/mark_read/	POST	Σήμανση συνομιλίας ως αναγνωσμένης
/api/conversations/{id}/	DELETE	Διαγραφή συνομιλίας για τον χρήστη που το ζητά
/api/categories/	GET	Οι κατηγορίες εκδηλώσεων
/api/admin/users/	GET	Οι χρήστες, για τον διαχειριστή
/api/admin/users/{id}/approve/	POST	Έγκριση αίτησης εγγραφής
/api/admin/users/{id}/reject/	POST	Απόρριψη αίτησης εγγραφής
/api/admin/events/export/xml/	GET	Εξαγωγή εκδηλώσεων ως xml
/api/admin/events/export/json/	GET	Εξαγωγή εκδηλώσεων ως json
/api/conversations/	POST	Έναρξη συνομιλίας για μια εκδήλωση
/api/messages/	POST	Αποστολή μηνύματος σε συνομιλία
/api/messages/{id}/	GET	Ανάγνωση μηνύματος, το οποίο σημαίνεται ως αναγνωσμένο
/api/messages/{id}/	DELETE	Διαγραφή μηνύματος για τον χρήστη που το ζητά
/api/messages/inbox/	GET	Εισερχόμενα μηνύματα
/api/messages/sent/	GET	Απεσταλμένα μηνύματα
/api/messages/unread_summary/	GET	Πλήθος αδιάβαστων, για την ένδειξη της μπάρας

5.3 Ταυτοποίηση και εξουσιοδότηση

Ο εξυπηρετητής δεν διατηρεί κατάσταση συνεδρίας. Η μόνη ενεργή κλάση ταυτοποίησης είναι η JWTAuthentication, οπότε κάθε αίτηση προς το API αναγνωρίζεται αποκλειστικά από την επικεφαλίδα Authorization και είναι απολύτως αυτοτελής. Η επιλογή αυτή ταιριάζει στην αρχιτεκτονική της εφαρμογής, όπου το μετωπιαίο άκρο εκτελείται σε διαφορετική προέλευση από το νωτιαίο.

Η σύνδεση γίνεται με αποστολή ονόματος χρήστη και κωδικού στο /api/auth/login/, η οποία επιστρέφει ένα ζεύγος tokens: ένα access token διάρκειας δέκα λεπτών και ένα refresh token διάρκειας μίας ημέρας. Έτσι εξασφαλίζεται η ασφάλεια και η ευχρηστία της εφαρμογής (access token περιορίζει το παράθυρο εκμετάλλευσης σε περίπτωση διαρροής, ενώ το refresh token επιτρέπει στον χρήστη να παραμείνει συνδεδεμένος χωρίς να ξαναδώσει συνθηματικό). Ο χειρισμός των tokens από την πλευρά του πελάτη περιγράφεται στο κεφάλαιο της ασφάλειας. Ο έλεγχος έγκρισης κατά τη σύνδεση και κατά την ανανέωση γίνεται από τους αντίστοιχους serializers οι οποίοι επεκτάθηκαν έτσι ώστε μετά την επιτυχημένη επαλήθευση από τον χρήστη να ελέγχεται και η κατάσταση έγκρισης του λογαριασμού από τον διαχειριστή.

5.4 Αναζήτηση και φιλτράρισμα

Το φιλτράρισμα, η αναζήτηση και η ταξινόμηση δεν υλοποιούνται με χειρόγραφο κώδικα μέσα στην όψη αλλά μέσω των τριών ενσωματωμένων μηχανισμών του DRF. Το DjangoFilterBackend χρησιμοποιείται για τα φίλτρα ισότητας και εύρους, το SearchFilter για την ελεύθερη αναζήτηση σε τίτλο και περιγραφή, και το OrderingFilter, για τη διάταξη κατά ημερομηνία έναρξης, χωρητικότητα ή ημερομηνία δημιουργίας.

5.4.1 Αναζήτηση και φιλτράρισμα εκδηλώσεων

Τα διαθέσιμα φίλτρα ορίζονται δηλωτικά στο EventFilter και είναι η κατηγορία, ο διοργανωτής, η κατάσταση, ο τύπος της εκδήλωσης, η πόλη, η χώρα, το χρονικό εύρος και το εύρος τιμών. Οι απαντήσεις που επιστρέφουν λίστες είναι σελιδοποιημένες με 30 εγγραφές ανά σελίδα. Η σελιδοποίηση είναι καθολική ρύθμιση και όχι επιλογή της κάθε όψης, ώστε καμία διαδρομή να μην μπορεί κατά λάθος να επιστρέψει ολόκληρο τον πίνακα.

5.5 Εκδηλώσεις

Η εκδήλωση έχει 4 καταστάσεις (DRAFT, CANCELLED, COMPLETED, PUBLISHED). Η κατάσταση της κάθε εκδήλωσης ορίζεται από τα services και είναι μόνο αναγνώσιμη στον πελάτη. Όταν μια εκδήλωση είναι στην κατάσταση DRAFT είναι ορατή μόνο στον δημιουργό της και μπορεί να προβεί σε όποια αλλαγή θέλει. Το επόμενο στάδιο είναι PUBLISHED, η εκδήλωση περνάει όλους τους ελέγχους του συστήματος και γίνεται ορατή σε όλους τους χρήστες. Με την πάροδο του χρόνου, όταν περάσει η ώρα λήξης, η κατάστασή της αλλάζει σε COMPLETED. Αυτό συμβαίνει αυτόματα από το σύστημα μέσω μιας μαζικής ενημέρωσης πριν από κάθε ανάγνωση του καταλόγου των εκδηλώσεων ή πριν από κάθε εξαγωγή δεδομένων από τον admin. Επιπλέον, υπάρχει και η κατάσταση CANCELLED. Σε αυτή την κατάσταση στέλνεται αυτόματα ένα τυποποιημένο μήνυμα σε κάθε χρήστη που έχει κάνει κράτηση στην εκδήλωση. Η εκδήλωση δεν διαγράφεται από τη βάση για λόγους ιστορικότητας και ιχνηλασιμότητας. Ο διοργανωτής μπορεί να προβεί σε διαγραφή εκδήλωσης (διαγραφή και από τη βάση δεδομένων) μόνο στην περίπτωση που αυτή βρίσκεται στην κατάσταση DRAFT ή αν δεν έχει γίνει καμία κράτηση.

5.5.1 Τύπος εισιτηρίων

Οι τύποι εισιτηρίων μπορούν να αλλάξουν όταν η εκδήλωση βρίσκεται στις καταστάσεις PUBLISHED, DRAFT και εφόσον δεν έχει ξεκινήσει (αν η ώρα εκκίνησης έχει περάσει δεν μπορεί να γίνει καμία αλλαγή στα εισιτήρια). Αυτό μπορεί να συμβεί υπό την προϋπόθεση πως τηρείται η συνολική χωρητικότητα, το πλήθος των διαθέσιμων θέσεων ανά τύπο εισιτηρίου και το συνολικό πλήθος δεσμευμένων θέσεων. Για να διαγραφεί ένας τύπος εισιτηρίου θα πρέπει να μην υπάρχει κάποια κράτηση με αυτό.

5.5.2 Φωτογραφίες

Ο διοργανωτής μπορεί να ανεβάσει και φωτογραφίες για την κάθε εκδήλωση (μέχρι 20). Ο τρόπος αποθήκευσής τους στη βάση είναι ο εξής: αποθηκεύεται σε αυτή το μονοπάτι ενώ η πραγματική φωτογραφία αποθηκεύεται στο τρέχον υπολογιστικό σύστημα κάτω από τον φάκελο media. Αυτό συμβαίνει καθώς η φωτογραφία πιθανώς θα είναι μεγάλη σε μέγεθος με αποτέλεσμα να γεμίζει τη βάση δεδομένων που χρησιμοποιούμε. Για να μην είναι χαώδης ο φάκελος media, οργανώνεται σε υποφακέλους με όνομα τον κωδικό της κάθε εκδήλωσης. Σε περίπτωση διαγραφής της εκδήλωσης διαγράφονται και οι φωτογραφίες από το τρέχον σύστημα. Τέλος, κάθε φωτογραφία ελέγχεται ως προς τη μορφή και το μέγεθός της: γίνονται δεκτές μόνο JPEG, PNG και WebP, με ανώτατο όριο τα 10 MB.

Επισημαίνεται ότι η βάση δεδομένων είναι κοινή, ενώ τα ίδια τα αρχεία των φωτογραφιών βρίσκονται στο υπολογιστικό σύστημα. **Επειδή υπάρχει όριο στο μέγεθος του αρχείου .zip που μπορεί να υποβληθεί στο eclass, υπάρχουν μόνο ενδεικτικά φωτογραφίες από έξι εκδηλώσεις στον φάκελο /media/.**

5.5.3 Κρατήσεις

Οι κρατήσεις μπορούν να γίνουν μόνο σε PUBLISHED εκδηλώσεις και ολοκληρώνονται σε δύο φάσεις. Στην πρώτη η κράτηση δημιουργείται σε κατάσταση PENDING, αφού ελεγχθεί ότι ο τύπος εισιτηρίου ανήκει στην εκδήλωση και ότι ο ζητούμενος αριθμός εισιτηρίων είναι διαθέσιμος (σε αυτό το στάδιο δεν δεσμεύεται καμία θέση). Στη δεύτερη δεσμεύονται τα εισιτήρια και η κράτηση περνά σε CONFIRMED, ενημερώνοντας τα διαθέσιμα εισιτήρια και τις συνολικές δεσμευμένες θέσεις. Ο διοργανωτής δεν μπορεί να κάνει κράτηση στη δική του εκδήλωση. Καμία κράτηση δεν τροποποιείται ούτε διαγράφεται μετά την υποβολή της. Η μόνη μετάβαση που μπορεί να γίνει είναι σε CANCELLED, ως συνέπεια ακύρωσης της εκδήλωσης από τον διοργανωτή, οπότε η εγγραφή διατηρείται για λόγους ιστορικότητας και ιχνηλασιμότητας. Τέλος, η κράτηση επιτρέπεται καθ' όλη τη διάρκεια της εκδήλωσης, μέχρι την ώρα λήξης της.

5.5.4 Επίλυση προβλήματος συνθηκών ανταγωνισμού

Το πρόβλημα εμφανίζεται όταν δύο χρήστες προσπαθούν ταυτόχρονα να κρατήσουν τα τελευταία εισιτήρια ενός τύπου. Χωρίς προστασία, και οι δύο διαβάζουν τα ίδια διαθέσιμα, και οι δύο βρίσκουν ότι επαρκούν, και η εκδήλωση καταλήγει με περισσότερες κρατήσεις από θέσεις. Η λύση είναι το κλείδωμα της γραμμής στη βάση. Κάθε ενέργεια που δεσμεύει εισιτήρια εκτελείται μέσα σε μία συναλλαγή (transaction) και ξεκινά με select_for_update() στη γραμμή του τύπου εισιτηρίου. Η δεύτερη αίτηση περιμένει μέχρι να ολοκληρωθεί η πρώτη και διαβάζει τα ενημερωμένα διαθέσιμα, οπότε απορρίπτεται κανονικά με μήνυμα εξάντλησης. Το ίδιο κλείδωμα χρησιμοποιείται και κατά τη διαγραφή εκδήλωσης, ώστε να μην μπορεί να διαγραφεί τη στιγμή που κάποιος ολοκληρώνει κράτηση σε αυτήν.

5.6 Διαχειριστής

Ο διαχειριστής μπορεί να προβεί σε όλες τις δυνατές ενέργειες της εφαρμογής. Αυτές είναι οι ενέργειες χρήστη, διαχείριση χρηστών, προβολή όλων των εκδηλώσεων σε οποιαδήποτε κατάσταση και αν βρίσκονται και η εξαγωγή δεδομένων σχετικά με τις εκδηλώσεις.

5.6.1 Εξαγωγή δεδομένων

Ο διαχειριστής μπορεί να εξάγει τα δεδομένα είτε σε μορφή XML, είτε σε μορφή JSON. Εξάγονται μόνο οι εκδηλώσεις που διαθέτουν τουλάχιστον μία κατηγορία και έναν τύπο εισιτηρίου, όπως απαιτεί το παρακάτω DTD:

```
<!ELEMENT Events (Event*)>
<!ELEMENT Event (Title, Category+, EventType, Venue, Address, City,
Country, GeoLocation?, StartDateTime, EndDateTime, Capacity,
TicketTypes, Bookings, Organizer, Status, Description, Media?)>
<!ATTLIST Event EventID CDATA #REQUIRED>
<!ELEMENT Title (#PCDATA)>
<!ELEMENT Category (#PCDATA)>
<!ELEMENT EventType (#PCDATA)>
```

```
<!ELEMENT Venue (#PCDATA)>
<!ELEMENT Address (#PCDATA)>
<!ELEMENT City (#PCDATA)>
<!ELEMENT Country (#PCDATA)>
<!ELEMENT GeoLocation EMPTY>
<!ATTLIST GeoLocation Latitude CDATA #REQUIRED Longitude CDATA #REQUIRED>
<!ELEMENT StartDateTime (#PCDATA)>
<!ELEMENT EndDateTime (#PCDATA)>
<!ELEMENT Capacity (#PCDATA)>
<!ELEMENT TicketTypes (TicketType+)>
<!ELEMENT TicketType (Name, Price, Quantity, Available)>
<!ATTLIST TicketType TicketTypeID CDATA #REQUIRED>
<!ELEMENT Name (#PCDATA)>
<!ELEMENT Price (#PCDATA)>
<!ELEMENT Quantity (#PCDATA)>
<!ELEMENT Available (#PCDATA)>
<!ELEMENT Bookings (Booking*)>
<!ELEMENT Booking (Attendee, Time, TicketTypeRef, NumberOfTickets,
TotalCost, BookingStatus)>
<!ATTLIST Booking BookingID CDATA #REQUIRED>
<!ELEMENT Attendee EMPTY>
<!ATTLIST Attendee UserID CDATA #REQUIRED>
<!ELEMENT Time (#PCDATA)>
<!ELEMENT TicketTypeRef (#PCDATA)>
<!ELEMENT NumberOfTickets (#PCDATA)>
<!ELEMENT TotalCost (#PCDATA)>
<!ELEMENT BookingStatus (#PCDATA)>
<!ELEMENT Organizer EMPTY>
<!ATTLIST Organizer UserID CDATA #REQUIRED>
<!ELEMENT Status (#PCDATA)>
<!ELEMENT Description (#PCDATA)>
<!ELEMENT Media (Photo*)>
<!ELEMENT Photo (#PCDATA)>
```

Σε διαφορετική περίπτωση τα δεδομένα των εκδηλώσεων δεν περιέχονται στο αρχείο που εξάγεται.

5.7 Μηνύματα

Στα μηνύματα όπως αναφέρεται και στην Παραδοχή 2.6 επιλέχθηκε να οργανωθούν σε συζητήσεις έτσι ώστε να μπορεί να στέλνεται στον χρήστη όλο το ιστορικό της συνομιλίας. Για τη διαγραφή συνομιλίας ή μηνυμάτων ακολουθείται η λογική soft-delete.

6 Σύστημα συστάσεων

Το σύστημα συστάσεων υλοποιεί τον αλγόριθμο Biased Matrix Factorization εκ του μηδενός, χωρίς χρήση έτοιμης βιβλιοθήκης μηχανικής μάθησης. Η πρόβλεψη της βαθμολογίας που θα έδινε ο χρήστης i στην εκδήλωση j είναι:

$$\boxed{\hat{x}_{ij} = \mu + b_i + c_j + \mathbf{v}_i \cdot \mathbf{f}_j}$$

όπου μ η καθολική μέση βαθμολογία, b_i και c_j οι μεροληψίες του χρήστη και της εκδήλωσης, και v_i, f_j τα λανθάνοντα διανύσματα. Η εκπαίδευση γίνεται με στοχαστική κατάβαση κλίσης μόνο πάνω στα γνωστά ζεύγη, ώστε οι κενές θέσεις του πίνακα να μη θεωρούνται μηδενικές βαθμολογίες. Σε κάθε εποχή τα δεδομένα ανακατεύονται, και η εκπαίδευση σταματά όταν το σφάλμα επικύρωσης δεν βελτιωθεί για δύο συνεχόμενες εποχές.

6.1 Βαθμολογίες

Ο αλγόριθμος απαιτεί αριθμητικές βαθμολογίες, οι οποίες όμως δεν δίνονται ρητά αφού η εφαρμογή δεν υποστηρίζει σύστημα βαθμολόγησης. Χρησιμοποιείται επομένως κοινή κλίμακα από 1 έως 5, η οποία τροφοδοτείται από δύο διαφορετικές πηγές. Η πρώτη πηγή είναι το σύνολο δεδομένων που δόθηκε στο eclass. Το αρχείο event_interest.csv δεν περιέχει βαθμολογίες αλλά δύο δυαδικές σημαίες, τις interested και not_interested, οι οποίες αντιστοιχίζονται ως εξής:

Σημαίες	Βαθμολογία
interested = 1	5.0
interested = not_interested = 0	3.0
not_interested = 1	1.0

Στην εφαρμογή οι χρήστες δεν βαθμολογούν εκδηλώσεις, οπότε η βαθμολογία προκύπτει έμμεσα από τη συμπεριφορά τους. Μια επιβεβαιωμένη κράτηση αντιστοιχεί σε βαθμολογία ίση με 5.0. Οι επαναλαμβανόμενες προβολές μιας εκδήλωσης αποτελούν ασθενέστερη ένδειξη και αντιστοιχούν σε βαθμολογία από 2.0 έως 3.0, αυξανόμενη κατά 0.5 ανά προβολή με ανώτατο όριο το 3.0. Όταν για το ίδιο ζεύγος υπάρχουν και τα δύο, η κράτηση υπερισχύει της προβολής.

Το σύνολο δεδομένων χρησιμοποιείται αποκλειστικά για την αξιολόγηση του αλγορίθμου, ενώ το μοντέλο που εξυπηρετεί τους χρήστες εκπαιδεύεται μόνο στα δεδομένα της εφαρμογής.

Το μοντέλο δεν εκπαιδεύεται σε κάθε αίτηση, γιατί η απόκριση θα καθυστερούσε πάρα πολύ. Διατηρείται στη μνήμη και ανακατασκευάζεται μόνο όταν αλλάξουν οι κρατήσεις ή οι προβολές, ή όταν περάσουν δέκα λεπτά από την τελευταία εκπαίδευση.

6.2 Αξιολόγηση

Η αξιολόγηση εκτελείται μέσω της εντολής python manage.py evaluate_recommender πάνω στο σύνολο δεδομένων event_interest.csv, το οποίο περιέχει 13.983 βαθμολογίες από 1.970 χρήστες για 8.119 εκδηλώσεις. Ο πίνακας παρουσιάζει πολλά κενά (πυκνότητα: 0.087%). Κατά την αξιολόγηση μετρώνται δύο διαφορετικά πράγματα. Το MAE και το RMSE δείχνουν πόσο κοντά είναι μια προβλεπόμενη βαθμολογία στην πραγματική, ενώ η Precision@K και η Recall@K δείχνουν αν οι εκδηλώσεις που πράγματι ενδιέφεραν τον χρήστη εμφανίζονται στην κορυφή της λίστας.

Τα αποτελέσματα είναι τα εξής:

Μετρική	Τιμή	Βάση σύγκρισης
MAE	0.7780	—
RMSE	0.9273	0.9965 (πρόβλεψη του μ)
Precision@5	0.01263	0.00019 (τυχαία επιλογή)
Recall@5	0.05402	—

Το RMSE είναι 6.9% χαμηλότερο από την πρόβλεψη της μέσης βαθμολογίας για κάθε ζεύγος, και η Precision@5 68 φορές καλύτερη από την τυχαία επιλογή. Οι τελικές τιμές είναι K = 2, learning_rate=0.01, REGULARIZATION=0.005 και MAX_EPOCHS = 17. Η εκπαίδευση συνήθως σταματά στις 7–9 epochs.

7 Μετωπιαίο άκρο (Frontend)

Στην παρούσα ενότητα περιγράφεται η οργάνωση και η δομή του μετωπιαίου άκρου της εφαρμογής EventHub. Οι τεχνολογίες που χρησιμοποιήθηκαν, οι μηχανισμοί ασφαλείας, οι σχεδιαστικές αποφάσεις και οι παραδοχές λειτουργίας έχουν ήδη τεκμηριωθεί στις Ενότητες 2 (Παραδοχές) και 3 (Ρόλοι). Εδώ παρουσιάζεται η αρχιτεκτονική οργάνωση του κώδικα, η δομή των φακέλων, τα βασικά components και η ροή δεδομένων.

7.1 Δομή του Frontend

Ο πηγαίος κώδικας του frontend είναι οργανωμένος σε τέσσερις βασικούς φακέλους, ακολουθώντας τη σύμβαση του React και διαχωρίζοντας τις αρμοδιότητες ανάλογα με το είδος του κώδικα:

```
src/
├── App.jsx
├── App.css
├── main.jsx
├── context/
├── components/
├── pages/
└── services/
```

Ρόλος κάθε φακέλου:

- **context**/: Περιέχει React Context providers για καθολική διαχείριση κατάστασης (authentication).
- **components**/: Περιέχει επαναχρησιμοποιήσιμα components που χρησιμοποιούνται σε πολλαπλές σελίδες.
- **pages**/: Περιέχει τα components-σελίδες, οργανωμένα σε υποφακέλους ανά λειτουργική περιοχή.
- **services**/: Περιέχει τη λογική επικοινωνίας με το backend API.

7.2 Δομή Φακέλων και Αρχείων

7.2.1 Root Level

Αρχείο	Περιγραφή
main.jsx	Σημείο εισόδου εφαρμογής
App.jsx	Κεντρικό routing και διάταξη σελίδων
App.css	Καθολικοί κανόνες στυλ

7.2.2 Context (src/context/)

Αρχείο	Περιγραφή
AuthContext.jsx	Provider για τη διαχείριση του authentication state. Παρέχει τις συναρτήσεις login, register, logout και το state user, isAuthenticated, loading.

7.2.3 Components (src/components/)

- Κοινά Components (components/common/):

Αρχείο	Περιγραφή
Navbar.jsx	Κύριο μενού πλοήγησης. Εμφανίζει διαφορετικές επιλογές ανάλογα με το αν ο χρήστης είναι συνδεδεμένος.
Footer.jsx	Υποσέλιδο της εφαρμογής.
LoadingSpinner.jsx	Ενιαίος spinner φόρτωσης, με δυνατότητα μηνύματος και full-height mode.
index.js	Barrel export των παραπάνω.

- Ειδικά Components:

Αρχείο	Περιγραφή
RequireRole.jsx	Wrapper για προστατευμένες διαδρομές βάσει ρόλου.
BookTicket.jsx	Modal επιλογής τύπου και ποσότητας εισιτηρίων.

7.2.4 Pages (src/pages/)

Ο φάκελος pages/ οργανώνεται σε υποφακέλους ανά λειτουργική περιοχή:

- Κύριες σελίδες (root του pages/):

Αρχείο	Περιγραφή
WelcomePage.jsx	Αρχική σελίδα καλωσορίσματος.
HomePage.jsx	Αρχική σελίδα με προβολή εκδηλώσεων ανά κατηγορία.

- Authentication (pages/auth/):

Αρχείο	Περιγραφή
SignIn.jsx	Φόρμα σύνδεσης.
SignUp.jsx	Φόρμα εγγραφής νέου χρήστη.

- Εκδηλώσεις (pages/events/):

Αρχείο	Περιγραφή
EventsPage.jsx	Σελίδα αναζήτησης και φιλτραρίσματος εκδηλώσεων.
EventDetails.jsx	Σελίδα αναλυτικών στοιχείων εκδήλωσης (καρουζέλ, χάρτης, tabs).
BookingPage.jsx	Σελίδα ολοκλήρωσης κράτησης.
RecommendationsPage.jsx	Σελίδα προτεινόμενων εκδηλώσεων.

- Προφίλ (pages/profile/):

Αρχείο	Περιγραφή
ProfilePage.jsx	Router-level σελίδα που επιλέγει ανάμεσα σε Admin και User dashboard.
UserDashboard.jsx	Dashboard απλού χρήστη με tabs (Personal, Messages, Bookings, Events, Reservations).
EditProfile.jsx	Φόρμα επεξεργασίας προσωπικών στοιχείων.

- Συμμετέχων (pages/profile/participant):

Αρχείο	Περιγραφή
MyBookings.jsx	Λίστα προσωπικών κρατήσεων με φίλτρα.

- Διοργανωτής (pages/profile/organizer):

Αρχείο	Περιγραφή
CreateEvent.jsx	Οδηγός τριών βημάτων για δημιουργία εκδήλωσης.
EditEvent.jsx	Οδηγός τριών βημάτων για επεξεργασία εκδήλωσης.
MyEvents.jsx	Λίστα εκδηλώσεων του χρήστη με φίλτρα και μαζική διαχείριση.
EventDetailsOrganizer.jsx	Αναλυτική προβολή εκδήλωσης από την πλευρά του διοργανωτή, με ενέργειες.
Reservations.jsx	Λίστα εκδηλώσεων με τον αριθμό κρατήσεων.
ReservationsDetails.jsx	Αναλυτική προβολή κρατήσεων ανά εκδήλωση.

- Διαχειριστής (pages/profile/admin):

Αρχείο	Περιγραφή
AdminDashboard.jsx	Dashboard διαχειριστή με tabs (Personal, Users, Events, Export).
UserManagement.jsx	Διαχείριση χρηστών με έγκριση/απόρριψη.
AdminEvents.jsx	Διαχείριση όλων των εκδηλώσεων.
ExportData.jsx	Εξαγωγή δεδομένων σε XML/JSON.

- Κοινά (pages/profile/common):

Αρχείο	Περιγραφή
NewMessage.jsx	Φόρμα δημιουργίας νέου μηνύματος.

7.2.5 Services (src/services)

Αρχείο	Περιγραφή
client.js	Κεντρικό Axios instance, με request/response interceptors για JWT.
api.js	Ορισμός όλων των API services (auth, event, category, booking, message, user).
helpers.js	Βοηθητικές συναρτήσεις (pagination fetching, ανάλυση κατηγοριών, συγχρονισμός εισιτηρίων και φωτογραφιών).

7.3 Κεντρικό Routing

Η δρομολόγηση της εφαρμογής υλοποιείται στο App.jsx μέσω του BrowserRouter της βιβλιοθήκης React Router v6. Η διάρθρωση των διαδρομών ακολουθεί την εξής λογική:

1. **Αυτόνομη διαδρομή** /: Αποδίδει τη WelcomePage χωρίς Navbar/Footer.
2. **Wildcard διαδρομή** *: Περιβάλλει όλες τις υπόλοιπες σελίδες με κοινό Navbar και Footer.
3. **Δημόσιες διαδρομές**: HomePage, SignIn, SignUp, EventsPage, EventDetails.
4. **Προστατευμένες διαδρομές**: Τυλίγονται με το component RequireRole και απαιτούν ταυτοποίηση.

Παράλληλα, το component PageTitle ενημερώνει δυναμικά τον τίτλο του browser tab βάσει της τρέχουσας διαδρομής.

7.4 Ροή Δεδομένων

Η ροή δεδομένων στην εφαρμογή ακολουθεί το εξής μοτίβο:

1. Ο χρήστης αλληλεπιδρά με κάποιο component σελίδας (π.χ. EventsPage).
2. Το component καλεί κάποια συνάρτηση από το layer των services (eventService.getEvents(...)).
3. Το service χρησιμοποιεί το κεντρικό Axios instance (api) για να επικοινωνήσει με το backend.
4. Ο Axios interceptor προσθέτει αυτόματα το JWT token και, αν χρειαστεί, ανανεώνει το access token.
5. Τα δεδομένα επιστρέφουν στο component, το οποίο ενημερώνει το τοπικό του state (useState) και ανα-αποδίδει το UI.

Για την καθολική κατάσταση του χρήστη, χρησιμοποιείται το AuthContext, το οποίο παρέχει το user και το isAuthenticated σε όλα τα components που τα χρειάζονται.

7.5 Δομή Διαδρομών ανά Σελίδα

Διαδρομή	Σελίδα	Πρόσβαση
/	Welcome Page	Δημόσια
/homepage	Home Page	Δημόσια
/signin	Sign In	Δημόσια
/signup	Sign Up	Δημόσια
/events	Events Page	Δημόσια
/events/details/:id	Event Details	Δημόσια
/events/details/:id/booking	Booking Page	User/Admin
/recommendations	Recommendations Page	User/Admin
/profile	Profile Page	User/Admin
/profile/edit	Edit Profile	User/Admin
/profile	User Dashboard (tab = personal/ messages/bookings/events/reservations)	User
/profile?tab=bookings	My Bookings	User
/profile?tab=events	My Events	User
/profile/events/new	Create Event	User
/profile/events/edit/:id	Edit Event	User
/profile/events/:id	Event Details Organizer	User
/profile?tab=reservations	Reservations	User
/profile/reservations/:id	Reservations Details	User
/profile/messages/new	New Message	User
/profile?tab=personal	Admin Dashboard	Admin
/profile?tab=users	User Management	Admin
/profile?tab=events	Admin Events	Admin
/profile?tab=export	Export Data	Admin

7.6 Επαναχρησιμοποιήσιμα Components

Η αρχιτεκτονική του frontend βασίζεται στη επαναχρησιμοποίηση components, ώστε να αποφεύγεται η επανάληψη κώδικα. Τα βασικότερα κοινά components είναι:

1. **Navbar**: Χρησιμοποιείται σε όλες τις σελίδες εκτός της WelcomePage. Προσαρμόζει το μενού του ανάλογα με το αν ο χρήστης είναι συνδεδεμένος, και εμφανίζει τον αριθμό των μη αναγνωσμένων μηνυμάτων.
2. **Footer**: Σταθερό υποσέλιδο σε όλες τις σελίδες εκτός της WelcomePage.
3. **LoadingSpinner**: Χρησιμοποιείται σε κάθε ασύγχρονη λειτουργία. Δέχεται δύο props (message και fullHeight) και προσαρμόζεται ανάλογα με το context.
4. **RequireRole**: Wrapper για προστατευμένες διαδρομές. Χρησιμοποιείται σε επίπεδο routing.
5. **BookTicket**: Modal που εμφανίζεται στη σελίδα EventDetails. Διαχειρίζεται την επιλογή τύπου και ποσότητας εισιτηρίων και οδηγεί στη BookingPage.

7.7 Διαχείριση Στυλ (CSS)

Η εφαρμογή χρησιμοποιεί custom CSS αντί για κάποια βιβλιοθήκη UI (π.χ. Tailwind, Bootstrap). Κάθε σελίδα ή component έχει το δικό του αρχείο .css, ενώ υπάρχει και ένα κοινό App.css για καθολικούς κανόνες. Η επιλογή αυτή προσφέρει:

- Πλήρη έλεγχο στην εμφάνιση.
- Αποφυγή εξάρτησης από εξωτερικές βιβλιοθήκες styling.
- Συνέπεια μέσω χρήσης κοινών CSS variables (χρώματα, διαστήματα).

Για τα εικονίδια χρησιμοποιείται η βιβλιοθήκη Material UI Icons, η οποία παρέχει έτοιμα SVG εικονίδια ως React components.

8 Ασφάλεια (TLS/SSL)

Στην εφαρμογή υπάρχουν δύο διακριτά κανάλια επικοινωνίας και αμφότερα εξυπηρετούνται μέσω TLS: το κανάλι μεταξύ του περιηγητή και του μετωπιαίου άκρου, και το κανάλι μεταξύ του μετωπιαίου και του νωτιαίου άκρου. Και οι δύο εξυπηρετητές διαπραγματεύονται TLS 1.3. Η εφαρμογή εκτελείται τοπικά και δεν διαθέτει δημόσιο όνομα (public URL). Για αυτό τον λόγο χρησιμοποιείται self-signed πιστοποιητικό που παράγεται κατά την εγκατάσταση του κώδικα από το script `scripts/make_certs.sh` με τη χρήση του openssl και έχει ισχύ 1 χρόνο. Ο εξυπηρετητής ανάπτυξης της Django δεν υποστηρίζει TLS, για αυτό χρησιμοποιείται η εντολή runserver_plus με παραμέτρους `--cert-file`, `--key-file` και `--threaded`. Ο εξυπηρετητής ανάπτυξης του Vite ρυθμίζεται μέσω του πεδίου server.https στο vite.config.js, όπου δηλώνεται το ζεύγος κλειδιού και πιστοποιητικού. Η θύρα καθορίζεται ρητά με strictPort, ώστε η διεύθυνση της εφαρμογής να παραμένει σταθερή και να συμφωνεί με τις διευθύνσεις που εμπιστεύεται το νωτιαίο άκρο.

Πέραν της ίδιας της κρυπτογράφησης, εφαρμόζονται οι ακόλουθες ρυθμίσεις. Τα cookies συνεδρίας και CSRF σημαίνονται ως Secure (SESSION_COOKIE_SECURE, CSRF_COOKIE_SECURE), ώστε ο περιηγητής να μην τα αποστέλλει ποτέ πάνω από μη κρυπτογραφημένη σύνδεση. Η ρύθμιση SECURE_SSL_REDIRECT ενεργοποιείται όταν η εφαρμογή δεν εκτελείται σε κατάσταση αποσφαλμάτωσης, ανακατευθύνοντας κάθε αίτηση HTTP σε HTTPS. Το ALLOWED_HOSTS ορίζεται ρητά αντί να αφεθεί ανοικτό. Τέλος, οι λίστες CORS_ALLOWED_ORIGINS και CSRF_TRUSTED_ORIGINS περιέχουν αποκλειστικά διευθύνσεις https, ώστε μια σελίδα που θα σερβιριστεί πάνω από HTTP να μην μπορεί να καταναλώσει το API.

Επειδή το πιστοποιητικό είναι self-signed και δεν προέρχεται από αναγνωρισμένη αρχή πιστοποίησης, ο περιηγητής εμφανίζει προειδοποίηση την πρώτη φορά. Η προειδοποίηση πρέπει να γίνει αποδεκτή στη διεύθυνση της εφαρμογής (https://localhost:5173) και στη διεύθυνση του API (https://localhost:8000/api). Σε διαφορετική περίπτωση ο περιηγητής απορρίπτει σιωπηλά τις αιτήσεις της εφαρμογής προς το API, χωρίς ορατό μήνυμα σφάλματος.

Όσον αφορά τα JWT περισσότερες πληροφορίες βρίσκονται στην Παραδοχή 2.2 και στο κεφάλαιο 5.3 (Ταυτοποίηση και Εξουσιοδότηση).

9 Εγκατάσταση και εκτέλεση

Προϋποθέσεις: python3 με venv (σε Ubuntu: sudo apt install python3-venv) και Node.js έκδοσης 24 ή νεότερης, το οποίο συνοδεύεται από το npm.

1. Ένα script δημιουργεί το virtual environment του νωτιαίου άκρου και εγκαθιστά τα πακέτα του (backend/requirements.txt), εγκαθιστά τα πακέτα του μετωπιαίου άκρου μέσω npm install, και τέλος παράγει το self-signed certificate με το private key του που χρησιμοποιείται για το TLS:

   ```
   sh ./scripts/install_requirements.sh
   ```

2. Πρέπει να αντιγραφούν τα .env αρχεία. Για την ευκολότερη αξιολόγηση υπάρχουν στους αντίστοιχους φακέλους αρχεία .env.example όμοια με τα .env οπότε το μόνο που απαιτείται είναι η αντιγραφή τους. Μέσα σε αυτά περιέχεται και η διεύθυνση βάσης (Neon) που χρησιμοποιήθηκε για την αποθήκευση των δεδομένων.

   ```
   cp backend/.env.example backend/.env
   cp frontend/.env.example frontend/.env
   ```

3. Απαιτούνται 2 τερματικά τα οποία πρέπει να παραμείνουν ανοιχτά.

   ```
   # 1st terminal for backend, port 8000
   cd backend  && source venv/bin/activate
   ```

```
python manage.py runserver_plus 8000 --threaded \
     --cert-file ../certs/localhost.pem --key-file ../certs/localhost-key.pem

# 2nd terminal for frontend, port 5173
cd frontend && npm run dev
```

4. Η εφαρμογή ανοίγει στο **https://localhost:5173**. Επειδή το πιστοποιητικό είναι self-signed ο περιηγητής εμφανίζει προειδοποίηση την πρώτη φορά. Αυτή πρέπει να γίνει αποδεκτή στο https://localhost:5173 αλλά και στο https://localhost:8000/api (advanced -> proceed). Σε διαφορετική περίπτωση ο browser θα μπλοκάρει τις αιτήσεις της εφαρμογής προς το API.

5. Από εκεί και πέρα ο χρήστης περιηγείται στο **https://localhost:5173**.

Στο αρχείο instructions.txt υπάρχουν αναλυτικότερες πληροφορίες. Όπου περιέχονται ενδεικτικοί λογαριασμοί (username/password) χρήστη και διαχειριστή.

10 Επίλογος

Η ανάπτυξη ξεκίνησε από τη βάση δεδομένων και τη διεπαφή REST και προχώρησε προς το μετωπιαίο άκρο, ώστε κάθε σελίδα να γράφεται πάνω σε ένα API που ήδη λειτουργούσε. Η επιχειρησιακή λογική συγκεντρώθηκε σε χωριστό επίπεδο υπηρεσιών, ώστε οι κανόνες που αφορούν περισσότερες από μία οντότητες να βρίσκονται σε ένα σημείο.

Η πιο απαιτητική δυσκολία ήταν οι συνθήκες ανταγωνισμού στις κρατήσεις καθώς έπρεπε να εξασφαλιστεί πως ο αριθμός των εισιτηρίων δε θα είναι παραπάνω από τη χωρητικότητα της εκδήλωσης. Στο σύστημα συστάσεων η δυσκολία δεν ήταν ο αλγόριθμος αλλά η μέτρησή του: η πρώτη αναζήτηση υπερπαραμέτρων συνέκρινε διαφορές μικρότερες από τη διακύμανση του ίδιου του σφάλματος, δηλαδή μετρούσε θόρυβο. Με συγκρίσεις κατά ζεύγη στους ίδιους διαχωρισμούς καταλήξαμε σε $K=2$, $\eta=0.01$, $\lambda=0.005$, με RMSE 0.9273 έναντι 0.9965 της πρόβλεψης του μέσου όρου.

Συνολικά, η εργασία μάς ανάγκασε να αντιμετωπίσουμε μια εφαρμογή ιστού ως σύστημα με ταυτόχρονους χρήστες και δεδομένα που πρέπει να μένουν συνεπή (με την επίλυση των race conditions).